



# Metodi speciali delle classi

*Python*

**In Python i metodi scritti con due trattini bassi prima e dopo il nome, come `__init__`, `__str__`, `__len__`, vengono chiamati metodi speciali o anche dunder methods, da “double underscore”.**

**Sono metodi che non vengono usati quasi mai chiamandoli direttamente, ma vengono richiamati automaticamente da Python in situazioni specifiche.**

**Servono per definire il comportamento degli oggetti quando vengono creati, stampati, confrontati, sommati, trasformati in testo, usati con `len()`, con gli operatori matematici o con altre funzioni standard del linguaggio.**



**In pratica, i metodi speciali permettono a una classe personalizzata di comportarsi come un tipo nativo di Python.**

**Per esempio, grazie a questi metodi posso fare in modo che un mio oggetto venga stampato bene con `print()`, possa essere confrontato con `==`, sommato con `+`, oppure possa restituire una lunghezza con `len()`.**



## Metodo `__init__`

**Il metodo `__init__` viene chiamato automaticamente quando viene creato un nuovo oggetto.**

**Serve per inizializzare gli attributi dell'oggetto.**

```
1. class Studente:
2.
3.     def __init__(self, nome, eta):
4.         self.nome = nome
5.         self.eta = eta
6.
7.
8. studente1 = Studente("Luca", 20)
9.
10. print(studente1.nome)
11. print(studente1.eta)
```

**Il metodo `__init__` riceve i dati passati al momento della creazione dell'oggetto e li salva dentro gli attributi della classe.**



**In questo caso "Luca" viene salvato in `self.nome`, mentre 20 viene salvato in `self.eta`, senza `__init__`, dovremmo assegnare questi valori manualmente dopo aver creato l'oggetto.**

## Metodo `__str__`

`__str__` stabilisce come deve essere rappresentato un oggetto quando viene convertito in stringa, ad esempio con `print()`.

```
1. class Studente:
2.
3.     def __init__(self, nome, eta):
4.         self.nome = nome
5.         self.eta = eta
6.
7.     def __str__(self):
8.         return "Studente: " + self.nome + ", età: " + str(self.eta)
9.
10.
11. studente1 = Studente("Luca", 20)
12.
13. print(studente1)
```

Quando scriviamo `print(studente1)`, Python cerca automaticamente il metodo `__str__`, se lo trova, usa il testo restituito da quel metodo.



Questo permette di stampare gli oggetti in modo leggibile, invece di ottenere una rappresentazione tecnica poco chiara.



## Metodo `__repr__`

Il metodo `__repr__` serve per fornire una rappresentazione più tecnica dell'oggetto. È pensato per sviluppatori e debugging.

```
1. class Studente:
2.
3.     def __init__(self, nome, eta):
4.         self.nome = nome
5.         self.eta = eta
6.
7.     def __repr__(self):
8.         return "Studente('" + self.nome + "', " + str(self.eta) + ")"
9.
10.
11. studente1 = Studente("Luca", 20)
12.
13. print(repr(studente1))
```

La differenza principale è che `__str__` è pensato per una stampa leggibile per l'utente, mentre `__repr__` è pensato per una rappresentazione più precisa e tecnica.

In molti casi `__repr__` mostra una forma simile al codice necessario per ricreare l'oggetto.



## Metodo `__len__`

Il metodo `__len__` viene chiamato quando usiamo la funzione `len()` su un oggetto.

```
1.class Squadra:
2.
3.     def __init__(self, nome, giocatori):
4.         self.nome = nome
5.         self.giocatori = giocatori
6.
7.     def __len__(self):
8.         return len(self.giocatori)
9.
10.
11.squadra1 = Squadra("Tigri", ["Luca", "Marco", "Anna"])
12.
13.print(len(squadra1))
```

In questo esempio la classe `Squadra` contiene una lista di giocatori. Grazie al metodo `__len__`, possiamo usare `len(squadra1)` come se l'oggetto fosse una lista.

Python richiama automaticamente `__len__` e restituisce il numero di giocatori.



## Metodo `__eq__`

Il metodo `__eq__` definisce il comportamento dell'operatore `==`.

```
1. class Prodotto:
2.
3.     def __init__(self, nome, prezzo):
4.         self.nome = nome
5.         self.prezzo = prezzo
6.
7.     def __eq__(self, altro):
8.         return self.nome == altro.nome and self.prezzo == altro.prezzo
9.
10.
11. prodotto1 = Prodotto("Mouse", 20)
12. prodotto2 = Prodotto("Mouse", 20)
13. prodotto3 = Prodotto("Tastiera", 35)
14.
15. print(prodotto1 == prodotto2)
16. print(prodotto1 == prodotto3)
```

Normalmente Python confronta due oggetti in base alla loro identità in memoria con `__eq__`, invece, possiamo decidere noi quando due oggetti devono essere considerati uguali.

In questo caso due prodotti sono uguali se hanno lo stesso nome e lo stesso prezzo.





## Metodo `__lt__`

`__lt__` definisce il comportamento dell'operatore `<`, cioè “minore di”.

```
1. class Prodotto:
2.
3.     def __init__(self, nome, prezzo):
4.         self.nome = nome
5.         self.prezzo = prezzo
6.
7.     def __lt__(self, altro):
8.         return self.prezzo < altro.prezzo
9.
10.
11. prodotto1 = Prodotto("Mouse", 20)
12. prodotto2 = Prodotto("Tastiera", 35)
13.
14. print(prodotto1 < prodotto2)
```

Il metodo `__lt__` permette di confrontare due oggetti usando l'operatore `<`.



In questo esempio il confronto viene fatto sul prezzo, quindi `prodotto1 < prodotto2` significa: “il prezzo del primo prodotto è minore del prezzo del secondo?”.

## Metodo `__add__`

Il metodo `__add__` definisce il comportamento dell'operatore `+`.

```
1. class Portafoglio:
2.
3.     def __init__(self, soldi):
4.         self.soldi = soldi
5.
6.     def __add__(self, altro):
7.         return Portafoglio(self.soldi + altro.soldi)
8.
9.     def __str__(self):
10.        return "Totale soldi: " + str(self.soldi)
11.
12.
13. portafoglio1 = Portafoglio(50)
14. portafoglio2 = Portafoglio(30)
15.
16. portafoglio3 = portafoglio1 + portafoglio2
17.
18. print(portafoglio3)
```

Quando scriviamo `portafoglio1 + portafoglio2`, Python richiama automaticamente il metodo `__add__`.

In questo caso somma i soldi dei due portafogli e crea un nuovo oggetto `Portafoglio` con il totale.



## Metodo `__contains__`

Il `__contains__` definisce il comportamento dell'operatore `in`.

```
1.class Zaino:
2.
3.    def __init__(self, oggetti):
4.        self.oggetti = oggetti
5.
6.    def __contains__(self, oggetto):
7.        return oggetto in self.oggetti
8.
9.
10.zaino1 = Zaino(["libro", "penna", "quaderno"])
11.
12.print("penna" in zaino1)
13.print("telefono" in zaino1)
```

Grazie a `__contains__`, possiamo usare `in` direttamente sull'oggetto.



In questo esempio Python controlla se un certo oggetto è presente nella lista interna `self.oggetti`.

## Metodo `__getitem__`

Il metodo `__getitem__` permette di accedere agli elementi di un oggetto usando le parentesi quadre.

```
1.class Registro:
2.
3.    def __init__(self, studenti):
4.        self.studenti = studenti
5.
6.    def __getitem__(self, indice):
7.        return self.studenti[indice]
8.
9.
10.registro1 = Registro(["Luca", "Anna", "Marco"])
11.
12.print(registro1[0])
13.print(registro1[1])
```

Con `__getitem__`, un oggetto può comportarsi come una lista o una struttura indicizzabile.

Quando scriviamo `registro1[0]`, Python richiama `__getitem__` e passa 0 come indice.



## Metodo `__setitem__`

Il metodo `__setitem__` permette di modificare un elemento usando le parentesi quadre.

```
1.class Registro:
2.
3.    def __init__(self, studenti):
4.        self.studenti = studenti
5.
6.    def __getitem__(self, indice):
7.        return self.studenti[indice]
8.
9.    def __setitem__(self, indice, valore):
10.        self.studenti[indice] = valore
11.
12.
13.registro1 = Registro(["Luca", "Anna", "Marco"])
14.
15.registro1[1] = "Giulia"
16.
17.print(registro1[1])
```



Il metodo `__setitem__` viene chiamato quando assegniamo un valore usando le parentesi quadre. In questo caso `registro1[1] = "Giulia"` sostituisce "Anna" con "Giulia".



## Metodo `__call__`

Il `__call__` permette di usare un oggetto come fosse una funzione.

```
1.class Moltiplicatore:
2.
3.     def __init__(self, numero):
4.         self.numero = numero
5.
6.     def __call__(self, valore):
7.         return valore * self.numero
8.
9.
10.doppio = Moltiplicatore(2)
11.
12.print(doppio(10))
```

In questo esempio l'oggetto `doppio` può essere chiamato come una funzione.



Quando scriviamo `doppio(10)`, Python richiama automaticamente il metodo `__call__`, passando 10 come valore.

## Metodo `__del__`

Il metodo `__del__` viene chiamato quando un oggetto viene eliminato dalla memoria.

Va usato con attenzione, perché non sempre è immediato prevedere quando Python eliminerà un oggetto.

```
1.class Connessione:
2.
3.     def __init__(self, nome):
4.         self.nome = nome
5.         print("Connessione creata")
6.
7.     def __del__(self):
8.         print("Connessione eliminata")
9.
10.
11.connessione1 = Connessione("Database")
12.
13.del connessione1
```



Il metodo `__del__` può essere usato per eseguire operazioni di pulizia, ma nella pratica moderna si preferiscono spesso altri strumenti, come i context manager con `with`, perché sono più controllabili e sicuri.

## Metodo `__enter__` e `__exit__`

I metodi `__enter__` e `__exit__` permettono a un oggetto di essere usato con il costrutto `with`.

```
1. class FileFinto:
2.
3.     def __enter__(self):
4.         print("Apro la risorsa")
5.         return self
6.
7.     def __exit__(self, tipo_errore, valore_errore, traceback):
8.         print("Chiudo la risorsa")
9.
10.
11. with FileFinto() as file:
12.     print("Uso la risorsa")
```

Il metodo `__enter__` viene eseguito all'inizio del blocco `with`, mentre `__exit__` viene eseguito alla fine, anche se c'è un errore.

Questo meccanismo è molto utile per gestire file o operazioni che devono essere chiuse correttamente.





## Metodo `__bool__`

**Il metodo `__bool__` stabilisce se un oggetto deve essere considerato True o False.**

```
1.class Carrello:
2.
3.     def __init__(self, prodotti):
4.         self.prodotti = prodotti
5.
6.     def __bool__(self):
7.         return len(self.prodotti) > 0
8.
9.
10.carrello1 = Carrello(["Mouse", "Tastiera"])
11.carrello2 = Carrello([])
12.
13.if carrello1:
14.    print("Il carrello contiene prodotti")
15.
16.if not carrello2:
17.    print("Il carrello è vuoto")
```

**In questo esempio un carrello viene considerato True se contiene almeno un prodotto, mentre viene considerato False se è vuoto.**

**Questo permette di usare gli oggetti direttamente nelle condizioni.**





| Metodo speciale           | Quando viene usato       | Esempio di uso                      |
|---------------------------|--------------------------|-------------------------------------|
| <code>__init__</code>     | Creazione dell'oggetto   | <code>Studente("Luca", 20)</code>   |
| <code>__str__</code>      | Stampa leggibile         | <code>print(oggetto)</code>         |
| <code>__repr__</code>     | Rappresentazione tecnica | <code>repr(oggetto)</code>          |
| <code>__len__</code>      | Lunghezza                | <code>len(oggetto)</code>           |
| <code>__eq__</code>       | Uguaglianza              | <code>oggetto1 == oggetto2</code>   |
| <code>__lt__</code>       | Minore di                | <code>oggetto1 &lt; oggetto2</code> |
| <code>__add__</code>      | Somma                    | <code>oggetto1 + oggetto2</code>    |
| <code>__contains__</code> | Presenza                 | <code>"x" in oggetto</code>         |
| <code>__getitem__</code>  | Accesso per indice       | <code>oggetto[0]</code>             |
| <code>__setitem__</code>  | Modifica per indice      | <code>oggetto[0] = valore</code>    |
| <code>__call__</code>     | Oggetto chiamabile       | <code>oggetto()</code>              |
| <code>__del__</code>      | Eliminazione oggetto     | <code>del oggetto</code>            |
| <code>__enter__</code>    | Inizio blocco with       | <code>with oggetto as x:</code>     |
| <code>__exit__</code>     | Fine blocco with         | <code>uscita dal with</code>        |
| <code>__bool__</code>     | Valore booleano          | <code>if oggetto:</code>            |

# Esempio completo:

```
1. class Videogioco:
2.
3.     def __init__(self, titolo, prezzo):
4.         self.titolo = titolo
5.         self.prezzo = prezzo
6.
7.     def __str__(self):
8.         return self.titolo + " costa " + str(self.prezzo) + " euro"
9.
10.    def __eq__(self, altro):
11.        return self.titolo == altro.titolo and self.prezzo == altro.prezzo
12.
13.    def __lt__(self, altro):
14.        return self.prezzo < altro.prezzo
15.
16.    def __add__(self, altro):
17.        return self.prezzo + altro.prezzo
18.
19.
20. gioco1 = Videogioco("Dark Quest", 30)
21. gioco2 = Videogioco("Space War", 50)
22. gioco3 = Videogioco("Dark Quest", 30)
23.
24. print(gioco1)
25. print(gioco1 == gioco3)
26. print(gioco1 < gioco2)
27. print(gioco1 + gioco2)
```



**Buon Davante a tutti**

